

火车订票系统设计报告

组长：张超超 学号：2023K8009906024

组员：朱首赫 学号：2023K8009906029

组员：冯曦熠 学号：2023K8009909006

0 分工说明

成员	负责模块
合作完成	系统架构设计、需求对照检查、集成测试
张超超	关系模式设计、核心查询功能实现、用户认证实现、订票与订单管理、管理员功能实现
朱首赫	代码审查与功能整合、项目结构规范化、ER 图绘制、范式细化分析、SQL 分析
冯曦熠	需求功能测试与验证、前端页面设计与美化、用户交互功能优化

1 总体设计说明

1.1 系统架构

本系统采用 Web 应用的三层结构，具体实现如下表所示：

表 1: 系统三层架构

层次	技术实现	说明
展示层	HTML + JavaScript	提供用户交互界面
中间层	C++ CGI	处理业务逻辑，连接数据库
数据管理层	PostgreSQL	存储所有业务数据

1.2 核心功能

系统实现了以下 9 个具体功能需求：

- 记录列车信息：**包括车次、站点、到发时间、票价。
- 记录座位情况：**支持座位在不同行程段内的复用。
- 乘客信息管理：**支持用户注册与登录。
- 查询车次：**支持通过具体车次号进行查询。
- 查询两地间车次：**支持直达路径及一次换乘路径的搜索。
- 查询返程信息：**基于当前查询快速切换出发与目的地。
- 预订车次座位：**实现订票业务逻辑。
- 管理订单：**支持历史订单查询与退票（取消订单）功能。
- 管理员界面：**提供全局业务数据的统计与监控。

2 数据处理说明

2.1 原始数据与预处理

系统使用 2026 年 3 月的全国高铁数据 (共计 8500+ 个车次)。通过 Python 脚本 `generate_import_files.py` 将原始 `.txt` 文件转换为 `.tbl` 文件, 最后利用 PostgreSQL 的 `\copy` 指令高效导入。

2.2 票价计算规则

由于原始票价为“始发站至当前站”的累计值, 非始发站间的票价采用差值计算:

$$\text{票价}_{A \rightarrow C} = \text{票价}_{\text{始发} \rightarrow C} - \text{票价}_{\text{始发} \rightarrow A}$$

本实验忽略卧铺信息, 仅处理二等座、一等座及商务座。

3 ER 图

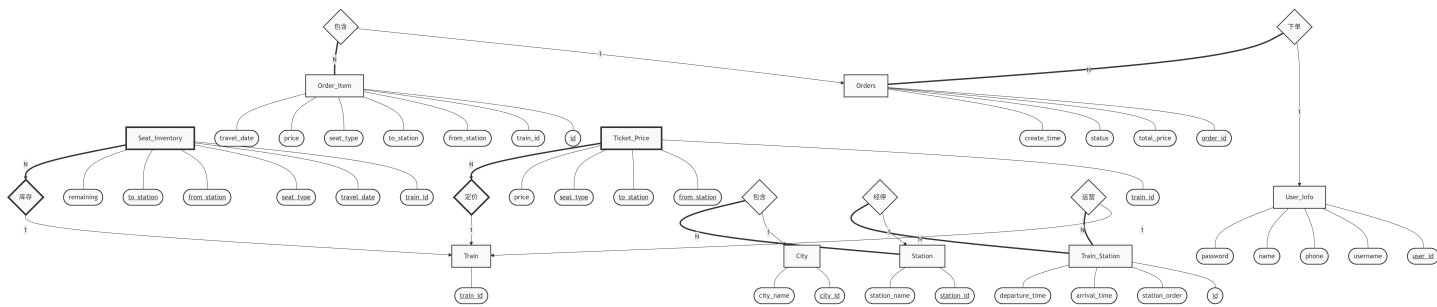


图 1: ER 图

4 关系模式设计

4.1 基础信息表

1. City (城市表)

列属性	类型	描述
<u>city_id</u>	SERIAL	城市编号 (主键)
city_name	VARCHAR(50)	城市名称 (唯一)

2. Station (车站表)

列属性	类型	描述
<u>station_id</u>	SERIAL	车站编号 (主键)
station_name	VARCHAR(20)	车站名称
city_id	INT	所属城市编号 (外键)

约束: UNIQUE(station_name, city_id)

3. Train (列车表)

列属性	类型	描述
<u>train_id</u>	VARCHAR(10)	车次编号（主键）

4.2 运营与票务表

4. Train_Station（列车经停信息表）

列属性	类型	描述
<u>id</u>	SERIAL	记录编号（主键）
train_id	VARCHAR(10)	车次编号（外键）
station_id	INT	车站编号（外键）
station_order	INT	该站在车次中的顺序
arrival_time	TIME	到达时间
departure_time	TIME	发车时间

约束：UNIQUE(train_id, station_order), UNIQUE(train_id, station_id)

5. Ticket_Price（票价表）

列属性	类型	描述
<u>train_id</u>	VARCHAR(10)	车次编号（联合主键）
<u>from_station</u>	INT	起始站编号（联合主键）
<u>to_station</u>	INT	终点站编号（联合主键）
<u>seat_type</u>	VARCHAR(20)	座位类型（联合主键）
price	DECIMAL(10,2)	票价

6. Seat_Inventory（座位库存表）

列属性	类型	描述
<u>train_id</u>	VARCHAR(10)	车次编号（联合主键）
<u>travel_date</u>	DATE	出行日期（联合主键）
<u>seat_type</u>	VARCHAR(20)	座位类型（联合主键）
<u>from_station</u>	INT	起始站编号（联合主键）
<u>to_station</u>	INT	终点站编号（联合主键）
remaining	INT	剩余座位数

4.3 用户与交易表

7. User_Info（用户表）

列属性	类型	描述
<u>user_id</u>	SERIAL	用户编号（主键）
username	VARCHAR(20)	用户名（唯一）
phone	VARCHAR(11)	手机号（唯一）
name	VARCHAR(20)	真实姓名
password	VARCHAR(100)	密码

8. Orders（订单表）

列属性	类型	描述
<u>order_id</u>	SERIAL	订单编号（主键）
user_id	INT	用户编号（外键）
total_price	DECIMAL(10,2)	订单总价
status	VARCHAR(10)	订单状态（正常/取消）
create_time	TIMESTAMP	下单时间

9. Order_Item（订单明细表）

列属性	类型	描述
<u>id</u>	SERIAL	记录编号（主键）
order_id	INT	订单编号（外键）
train_id	VARCHAR(10)	车次编号
from_station	INT	起始站编号
to_station	INT	终点站编号
seat_type	VARCHAR(20)	座位类型
price	DECIMAL(10,2)	该段票价
travel_date	DATE	出行日期

5 范式细化分析

本节对数据库核心表进行函数依赖（FD）分析，并基于判定条件证明其规范化程度。

5.1 基础地理信息：City 与 Station

- **City 表**：候选键为 `city_id` 或 `city_name`。决定因素均为候选键，满足 BCNF。
- **Station 表**：候选键为 `station_id` 或 `(station_name, city_id)`。每一个行列决定因素都是候选键，不存在传递依赖，满足 BCNF。

5.2 运行逻辑：Train_Station 表

候选键为 `id`、`(train_id, station_order)` 或 `(train_id, station_id)`。决定因素均为候选键，不存在非主属性间的传递依赖，满足 BCNF。

5.3 定价与库存: Ticket_Price 与 Seat_Inventory

这两张表作为关联实体, 其属性对候选键具有完全函数依赖关系。票价和库存均由其复合主键唯一确定, 满足 BCNF。

5.4 用户与订单模块

User_Info、Orders 及 Order_Item 的所有非键属性均直接依赖于主键, 无部分或传递依赖, 均满足 BCNF。

6 核心功能 SQL 实现

本节不逐行列出源码中的 SQL, 而是按功能需求说明系统使用的 SQL 类型、完成的数据库操作以及对应的关系代数思想。为简化表示, 记 City 为 C , Station 为 S , Train 为 T , Train_Station 为 TS , Ticket_Price 为 TP , Seat_Inventory 为 SI , User_Info 为 U , Orders 为 O , Order_Item 为 OI 。

6.1 需求 1: 记录列车信息

该需求主要由建表约束和数据导入操作完成。系统使用 CREATE TABLE 定义 Train、Station、Train_Station 和 Ticket_Price 等关系表, 并通过 COPY 导入预处理后的车次、车站、经停顺序、到发时间和票价数据。

- 车次编号由 T 保存; 车站名称、所属城市由 $S \bowtie C$ 表示, 其中 station_name 使用 VARCHAR(20) 满足站名长度约束。
- 列车经停详情由 TS 保存, UNIQUE(train_id, station_order) 保证同一车次站序唯一, UNIQUE(train_id, station_id) 保证同一车站在同一车次中不重复出现。
- 票价由 TP 保存, 主键为车次、起点站、终点站和席别的组合, 系统用 seat_type 区分二等座、一等座和商务座。
- 列车每天准时运行、票价固定不变的假设体现在: 时刻表和票价表作为静态基础数据导入后不随日期更新, 日期只影响库存和订单。

查询某车次完整经停信息时, 对 TS 、 S 、 C 做连接, 并按站序排序:

$$R_{train} = \pi_{order, city, station, arrival, departure}(\sigma_{train=t}(TS) \bowtie S \bowtie C)$$

6.2 需求 2: 记录座位情况

该需求由 Seat_Inventory 表和库存初始化、库存扣减两类操作完成。库存表的主键包含车次、日期、席别、起点站和终点站, 因此库存不是按整趟车保存, 而是按区间保存。

- 初始化库存时, 系统对指定车次和日期执行 INSERT ... SELECT: 从 TP 中取出该车所有可售区间和席别, 插入到 SI 中, remaining 初始值为 5; 若记录已存在, 则通过 ON CONFLICT DO NOTHING 保持原库存。
- 查询余票时, 若某日期尚未显式生成库存记录, 则用 COALESCE 将余票视为默认值 5。
- 座位共享通过区间库存实现。一次订票只影响与所购区间重叠的库存区间; 不重叠区间不会被扣减, 因此同一座位可在前一乘客下车后继续售卖给后续区间乘客。

设用户购买区间为 (a, b) , 库存区间为 (x, y) , 二者重叠条件为:

$$x < b \wedge a < y$$

订票时需要锁定并扣减的库存集合为:

$$I_{overlap} = \sigma_{x < b \wedge a < y}(SI \bowtie TS_x \bowtie TS_y)$$

6.3 需求 3: 乘客信息管理

注册和登录分别由 `register.cgi` 与 `login.cgi` 完成。

- 注册前先在 U 上执行选择和聚合检查：按用户名统计是否已存在、按手机号统计是否已存在；手机号还在程序层检查是否为 11 位数字。
- 若用户名已存在，当前实现执行更新操作，只修改该用户名对应用户的密码；若用户名不存在且手机号未占用，则向 U 插入新元组。
- 登录时在 U 上按用户名选择用户记录，取出用户编号、真实姓名和密码；程序比较密码后，普通用户进入首页，管理员用户跳转到管理界面。
- 首页再次通过用户名查询用户编号，确认用户存在后展示车次查询、线路查询和订单管理入口。

注册检查可表示为：

$$\text{count}(\sigma_{\text{username}=u}(U)), \quad \text{count}(\sigma_{\text{phone}=p}(U))$$

登录验证可表示为：

$$\pi_{\text{user_id,name,password}}(\sigma_{\text{username}=u}(U))$$

6.4 需求 4: 查询车次

单车次查询由 `query_train.cgi` 完成，输入为车次号和日期。程序首先检查日期必须晚于当天，然后为该车次和日期初始化库存。

- 席别查询：从 TP 中选择该车次存在的所有 `seat_type`，并按商务座、一等座、二等座排序。
- 静态信息查询：对 TS 、 S 、 C 进行连接，得到每站城市、站名、到达时间、发车时间，并按 `station_order` 排序。
- 票价查询：先取该车次始发站，再将 TP 与当前站连接，得到“始发站到当前站”的对应席别票价。
- 动态余票查询：在 SI 中按车次、日期和席别选择库存记录，并对覆盖当前到达站的库存取最小值，作为该站可展示余票。
- 跳转订票：若当前站不是始发站、票价存在且余票大于 0，则生成到 `book.cgi` 的订票参数；页面还提供任意起终站区间选择表单。

核心关系运算为：

$$R_{\text{stop}} = \pi_{\text{order,city,station,arr,dep,fare,left}}(\sigma_{\text{train}=t}(TS) \bowtie S \bowtie C \bowtie TP \bowtie SI)$$

其中 `fare` 来自票价关系 TP ，`left` 来自库存关系 SI 的聚合最小值。

6.5 需求 5: 查询两地间车次

两地查询由 `query_route.cgi` 完成。程序先将出发城市名、到达城市名转换为城市编号，避免后续查询反复按城市名连接；同时支持可选的出发站和到达站精确过滤。

直达查询：系统从 TP 出发，连接起点站、终点站以及两端在 TS 中的时刻记录，筛选出发城市、到达城市、发车时间和可选站点均满足条件的票价区间；再左连接 SI 获得余票，只保留余票大于 0 的结果。结果在程序中按车次和时间分组，并为商务座、一等座、二等座分别展示票价、余票和购票入口。

直达候选可表示为：

$$D = \sigma_{city_f=c_f \wedge city_t=c_t \wedge dep \geq time \wedge order_f < order_t}(TP \bowtie S_f \bowtie S_t \bowtie TS_f \bowtie TS_t \bowtie SI)$$

中转查询：系统使用多阶段关系运算生成一次换乘方案。

1. 第一程候选 `first_leg`：从出发城市到非出发、非到达城市的中转站，要求发车时间不早于输入时间。
2. 第二程候选 `second_leg`：从同一中转城市出发到目标城市，要求到达站在第二程出发站之后。
3. 候选组合 `transfer_candidates`：连接第一程和第二程，要求两段席别相同、车次不同、中转城市相同。
4. 可用性过滤 `available_candidates`：分别连接两段库存，取两段余票最小值作为总余票，并过滤无票方案。
5. 排名去重 `ranked_candidates`：对同一组“第一程车次、第二程车次、席别”按总价排名，只保留价格最低的方案。

中转换乘时间约束为：同站换乘等待 1-4 小时；异站但同城换乘等待 2-4 小时。可表示为：

$$Transfer = \sigma_{sameCity \wedge train_1 \neq train_2 \wedge seat_1 = seat_2 \wedge timeWindow}(FirstLeg \bowtie SecondLeg)$$

其中：

$$timeWindow = (sameStation \wedge 1h \leq wait \leq 4h) \vee (\neg sameStation \wedge 2h \leq wait \leq 4h)$$

中转结果由程序分页展示，每页最多 20 个中转组合。

6.6 需求 6：查询返程信息

返程查询不需要新增数据库关系或新的 SQL 模板，而是复用需求 5 的两地查询逻辑。页面在正向查询结果末尾生成返程链接，将出发城市和到达城市参数对调，并附带日期、时间等查询参数；用户点击后再次执行需求 5 中的直达查询和中转查询。

从关系运算角度看，返程只是将选择条件中的城市参数交换：

$$Route(c_f, c_t, d, t) \Rightarrow Route(c_t, c_f, d', t')$$

6.7 需求 7：预订车次座位

订票由 `book.cgi` 完成，分为确认页和正式提交两步。正式提交时使用事务保证订单、订单明细和库存更新的一致性。

- 费用计算：先从 `TP` 中查询所选车次、区间和席别的票价，并连接 `S` 获得起终站名称；单程总价为票价加 5 元订票费，换乘订单总价为两段票价之和加 10 元订票费。
- 事务开始后，先锁定当前用户记录，防止同一用户并发提交多个相互冲突的订单。
- 时间冲突检查：构造新订单行程时间区间，并与该用户已有正常订单的每段行程区间比较；若满足“已有出发时间早于新到达时间，且新出发时间早于已有到达时间”，则判定为时间重叠并回滚。
- 库存检查：对购买区间对应的重叠库存记录执行 `SELECT ... FOR UPDATE`，取最小余票判断是否仍可售。
- 库存更新：对所有与购买区间重叠的 `SI` 记录执行扣减操作。
- 订单生成：向 `O` 插入订单主表记录并返回订单号；单程插入一条 `OI` 明细，换乘插入两条 `OI` 明细；全部成功后提交事务，否则回滚。

时间冲突检查可表示为：

$$Conflict = \exists e \in ExistingOrders, (e_{dep} < n_{arr}) \wedge (n_{dep} < e_{arr})$$

库存扣减对象仍为需求 2 中的 $I_{overlap}$ 。

6.8 需求 8：管理订单

订单管理由 `orders.cgi` 完成，包括日期范围查询、订单详情和取消订单。

- 订单列表查询：在 O 中按当前用户编号和创建日期范围选择订单；通过横向子查询聚合 OI 中的车次链和席别链；再取第一段起点、最后一段终点，并连接 S 和 TS 得到起终点站名、首段出发时间和末段到达时间。
- 订单详情查询：连接 OI 、 O 、 S 和 TS ，展开订单中每个程段的车次、起终点、乘车日期、到发时间、席别和价格。
- 取消订单：事务内先按订单号和用户编号锁定订单，确认状态为“正常”；再遍历订单明细，对每段对应的库存执行加一操作；最后将订单状态更新为“取消”并提交。

订单列表的核心关系运算为：

$$R_{orders} = \pi_{order, date, trains, seats, from, to, dep, arr, total, status}(\sigma_{user=u \wedge date \in [l, r]}(O) \bowtie OI \bowtie S \bowtie TS)$$

取消订单的更新逻辑可概括为：

$$O.status \leftarrow \text{取消}, \quad SI.remaining \leftarrow SI.remaining + 1$$

6.9 需求 9：管理员界面

管理员界面由 `admin.cgi` 完成。登录时系统会确保固定管理员账户 `admin` 存在；访问管理界面前通过用户名判断是否为管理员。

- 总订单数：对 O 中状态为“正常”的订单执行计数聚合。
- 总成交金额：对 O 中状态为“正常”的订单执行总价求和聚合。
- 热点车次排行：将 OI 与 O 连接，只统计正常订单中的车次订票次数；再与 T 左连接，使没有订单的车次也可参与排序；最终按订票数降序、车次号升序取前 10。
- 注册用户列表：按用户编号顺序投影 U 中的用户名、真实姓名和手机号。
- 指定用户订单查看：按用户名连接 U 与 O ，并通过横向聚合查询该用户订单中的车次链、席别链、首段出发时间和末段到达时间。

热点车次统计对应的聚合关系运算为：

$$HotTrain = \tau_{cnt \downarrow}(\gamma_{train; count(*) \rightarrow cnt}(\sigma_{status=normal}(O) \bowtie OI))$$

取排序后的前 10 条即为管理员看板中的热门车次排行。